Steganografi Sunumu: Slayt Slayt Anlatım Rehberi ve Hoca Soruları

Sunum dosyası: 31 slayt. Her slayt için üç bölüm var: **Önce sen anla**, **Sunumda söyle**, **Hoca sorabilir**. Kaynaklar sunumdakiyle aynı: [1] Subramanian vd. 2021, [2] Fridrich vd. 2007, [3] Bao vd. 2018, [4] Fan vd. 2011.

Ana hikâye: Bir yöntem gelir, bir zayıflığı yüzünden yakalanır, sonraki yöntem o zayıflığı düzeltir. LSB, JSteg, Outguess, F3, F4, F5, matrix embedding, wet paper codes/STC, JPEG sıkıştırmaya dayanıklılık.

Dürüstlük notu: hangi sayı nereden?

- **Makalede birebir var:** Fan vd. (1,7,3,2) örneği ve hash hesapları [4]; genişletme katmanları tablosu [4]; (n,k) verimlilik tablosu [4]; 3×7 H matrisi [2]; p ve α_p eşlemesi [2]; hata olasılığı tablosu [2]; STC hata yayılımı formülleri [3]; Bao'nun yazma-geri kuralı ve eşikleri [3].
- **Kural makalede, sayıları ben seçtim (örnek amaçlı):** LSB pikselleri (150, 201, 78); anahtarlı LSB örneği (konumlar 5,2,7,0); DCT/kuantizasyon örneği (52,3 ve adım 10); JSteg dizisi (6,0,1,7,4,2); F3 ve F4 örnekleri; F5 shrinkage dizisi (1,3,4); matrix embedding (x=1011001, m=011); (1,3,2) hücresi (a=101, w=11). Hocaya şunu söyleyebilirsin: "Kural makalede, sayıları örnek olsun diye ben seçtim."
- **Makale dışı bilgi:** Hamming(7,4) adı ve tek hata düzeltme özelliği genel kodlama teorisidir. Atıf: R. W. Hamming, "Error detecting and error correcting codes", Bell System Technical Journal 29(2), 1950. Ayrıca JSteg'de "çiftlerin eşitlenmesi" mekanizması genel steganaliz bilgisidir; makalelerde sadece "POV problemi χ^2 testiyle yakalanır" yazar [4].
- **Kontrol et:** F1/F2 bu makalelerde geçmiyor, anlatım F3'ten başlıyor [4]. Hoca "F1/F2" derse neyi kastettiğini sor.

Slayt 1: Kapak

Önce sen anla. Bu bir literatür taraması: LSB'den F5'e fikirler nasıl gelişti ve nerede tıkandı.

Sunumda söyle. "LSB'den başlayıp JPEG tabanlı F5'e ve matrix embedding'e kadar gelen yöntemleri, nasıl çalıştıklarını ve neden yetersiz kaldıklarını anlatacağım. Her yöntemi küçük bir örnekle göstereceğim."

Hoca sorabilir.

- *Tezinle bağlantısı ne?* Tezim steganografi ve cebirsel kodlama teorisi; matrix embedding bu iki alanın kesişimi (Hamming kodlarıyla gömme).

Slayt 2: Hiding the message itself

Önce sen anla. Kriptografi içeriği saklar, steganografi varlığı. Steganografi, mesaj okunmasa bile varlığı anlaşılırsa "kırılmış" sayılır [2], [4]. Gömme: y = Emb(x, m, k); çıkarma: Ext(y, k) = m [2].

Sunumda söyle. "Şifreli metin ortada görünür ve şüphe çeker [1]. Steganografide amaç iletişimin varlığını gizlemek. Gömme cover, mesaj ve anahtardan stego üretir; çıkarma aynı anahtarla mesajı geri verir."

Hoca sorabilir.

- *Neden "varlığı tespit edilirse kırıldı" deniyor?* Hedef şüphe çekmeden iletişim; şüphe oluştuysa mesaj okunmasa bile amaç kaybolmuştur [2], [4].
- *Watermarking'den farkı?* Steganografi gizliliğe, watermarking dayanıklılığa vurgu yapar [1].
- *İkisi birlikte kullanılır mı?* Evet; [1]'de LSB gömmesini kriptografiyle birleştiren çalışmalar var.

Slayt 3: The embedding pipeline, step by step

Önce sen anla. Her sistem aynı iskelete sahip: cover seç, mesajı bite çevir (gerekirse hata düzeltme ekle), alan seç, konum seç, gömme işlemini uygula, gönder. Alıcı aynı anahtar/kodla çıkarır. Yöntemleri ayıran 3., 4. ve 5. adımlardır [2], [3], [4].

Sunumda söyle. "Göreceğimiz her yöntem bu hattın bir adımını değiştiriyor: JSteg alanı DCT'ye taşıyor, F5 gömme işlemini ve kodlamayı değiştiriyor, STC konum seçimini maliyete bağlıyor."

Hoca sorabilir.

- *Kapasite ile verimlilik farkı?* Kapasite toplam gömülebilen bit (log2|M|); verimlilik birim bozulma/değişiklik başına bit [2], [4].
- *Ölçütler?* Kapasite bpp = L/(H·W·C) [1] veya JPEG'de bpac [2]; kalite PSNR = 10·log10(R^2/MSE) [1]; güvenlik tespit hatası P [2].

Slayt 4: LSB: simple, and easy to catch

Önce sen anla. Piksellerin son bitini mesaj bitiyle değiştirirsin. Rastgele verilerde bir pozisyon yarı olasılıkla zaten doğrudur, yani %50 değişir; verimlilik değişiklik başına 2 bit. Resmi doldurursan verinin yarısı değişir ve istatistiksel olarak yakalanır [4]. Zayıflıklar: aşırı yükleme görünür iz bırakır [1]; spatial yöntemler piksel değişimine ve görsel saldırıya açıktır [4]; LSB çevirmenin kendisi tespit edilebilir [2]; JPEG yeniden sıkıştırma mesajı bozar [3].

Sunumda söyle. "LSB basit ve kapasitesi yüksek ama güvenliği zayıf. Kayıplı kanaldan geçerse mesaj kaybolur; sosyal medyada işe yaramaz."

Hoca sorabilir.

- *Neden 2 bit/değişiklik?* Mesaj ve cover bitleri düzgün dağılımlıysa gömülecek bit yarı olasılıkla zaten aynı; ortalama yarısı değişir [4].
- *Varyantlar?* k-LSB, Huffman+LSB, RGB için 2:2:4, LSB+PVD, anahtarlı rastgele konum [1].

Slayt 5: LSB by example

Önce sen anla. Üç piksel: 150 (10010110), 201 (11001001), 78 (01001110). Mesaj 1, 0, 1. Sadece son bit değişir: 151, 200, 79. Alıcı son bitleri okur: 1, 0, 1. Her değer en fazla 1 oynar.

Sunumda söyle. "Üç piksele üç bit gömüyorum. Göz 1 birimlik değişimi fark etmiyor ama değişikliklerin istatistiksel kalıbı steganaliz için ipucu." (Sayılar örnek, kural [1]'den.)

Hoca sorabilir.

- *Piksel zaten istenen bitteyse?* Değişiklik yapılmaz.
- *Sıralı mı rastgele mi?* İkisi de olur; anahtarlı rastgele yol bir sonraki slaytta [1], [2].

Slayt 6: Key-driven LSB (dynamic key), step by step

Önce sen anla. Gönderici ve alıcı bir anahtar paylaşır. Anahtarla başlatılan bir PRNG pikseller için bir sıra üretir (örnekte 5, 2, 7, 0). Bitler bu sırayla gömülür: pixel 5 = 100 + bit 1 → 101; pixel 2 = 57 + bit 0 → 56; pixel 7 = 210 + bit 1 → 211; pixel 0 = 33 + bit 1 → aynı. Alıcı aynı anahtardan aynı sırayı üretir ve LSB'leri okur: 1, 0, 1, 1.

Sunumda söyle. "Konumlar sabit bir tarama yerine anahtara bağlı. Ama LSB çevirmenin kendisi hâlâ iz bırakıyor [2]." Kaynak: Patel & Meena çalışması olarak [1]'de geçer.

Hoca sorabilir.

- *Dynamic LSB tam olarak ne?* Makalelerde tek bir standart algoritma olarak tanımlı değil. [1]'de PRNG ile piksel seçimi ve anahtarın döndürmeyle şifrelenmesi anlatılıyor; ben "anahtara bağlı rastgele konum" fikri olarak anlattım.
- *JSteg ile benzerliği?* JSteg de sıralı yoldan sözde-rastgele yola geçti [2]; Outguess de PRNG ile saçıyor [4].

Slayt 7: Spatial vs transform domain

Önce sen anla. Spatial: piksel LSB'leri. Transform: JPEG'in 8×8 blok DCT katsayıları. JPEG hattı: renk dönüşümü, alt örnekleme, 8×8 bloklar, DCT, kuantizasyon, kodlama [3]. Gömme kuantize edilmiş katsayılara yapılır. F5 sadece sıfır olmayan AC katsayılarına gömer [2].

Sunumda söyle. "Piksel alanı piksel değişimine ve görsel saldırıya açık [4]. JPEG en yaygın format olduğu için çalışmalar DCT katsayılarına gömmeye yoğunlaştı [2], [4]."

Hoca sorabilir.

- *DC ve AC nedir?* DC blok ortalaması, 63 AC katsayı ayrıntıyı taşır; MBS 63 AC modu için model kurar [2].
- *Kuantizasyon neden önemli?* Katsayılar tam sayıya yuvarlanır; gömme bu tam sayılara yapılır [3].

Slayt 8: From pixels to the coefficient we embed into

Önce sen anla. 8×8 piksel bloğu DCT'den geçer (1 DC + 63 AC). Diyelim bir AC katsayısı 52,3 çıktı. Kalite faktörüne bağlı adım (örnek 10) ile bölünür: 52,3 / 10 = 5,23 → yuvarlanıp 5. Saklanan ve steganografinin değiştirdiği şey bu 5. Örnek: F3 kuralıyla bit 0 gelirse 4 olur. Decoder aynı adımla çarpar: 5 → 50, 4 → 40. Kayıplı yeniden sıkıştırma farklı QF ile bu tam sayıları değiştirir [3].

Sunumda söyle. "Gömme DCT'nin ham değerine değil, kuantize edilmiş tam sayıya yapılıyor. Bu yüzden yeniden sıkıştırma mesajı bozabiliyor." (Sayılar örnek; hat [3]'ten.)

Hoca sorabilir.

- *Neden kuantize katsayıya gömülüyor?* Dosyada saklanan değerler bunlar; adaptif JPEG yöntemleri bu yuvarlanmış katsayıları değiştirir [3].
- *QF ne demek?* Kuantizasyon adımlarını belirleyen kalite faktörü [3].

Slayt 9: From JSteg to F5 (özet tablo)

Önce sen anla. Bu slayt haritadır: beş yöntemin tek cümlelik fikri ve sorunu. Detaylar sonraki slaytlarda.

Sunumda söyle. "Önce haritayı göstereyim, sonra her yöntemi örnekle açıyorum."

Hoca sorabilir.

- *Neden bu sıra?* Kronolojik ve her biri öncekinin zayıflığını düzeltiyor [4].

Slayt 10: JSteg: LSB inside JPEG

Önce sen anla. JSteg, LSB'nin JPEG sürümü. 0 ve 1 değerli katsayılar atlanır: bunlar bir LSB çiftidir ve 0'ı değiştirmek büyük bozulma yaratır [2]. Yol önce sıralı, sonra sözde-rastgele [2]. Sorun: DCT histogramında karakteristik izler [2]; POV problemi χ^2 testiyle yakalanır [4]; çok düşük oranlarda (0.05 bit/kullanılabilir katsayı) bile category attack ile bulunur [2].

Sunumda söyle. "JSteg bir LSB kopyası, tek fark alanın DCT katsayıları olması. Histogramın birinci derece istatistiği bozuluyor ve basit testlerle yakalanıyor."

Hoca sorabilir.

- *χ^2 testi nasıl çalışır?* Makalelerde detay yok; sadece POV sorununu yakaladığı yazıyor [4]. Fikir genel steganaliz bilgisi: ardışık iki değerin sayıları gömme sonrası anormal yakınlaşır.
- *Neden 0 ve 1 atlanıyor?* [2]'ye göre aralarında LSB çifti var ve 0'a dokunmak büyük bozulma getirir.

Slayt 11: JSteg, step by step

Önce sen anla. Katsayılar: 6, 0, 1, 7, 4, 2; mesaj: 1, 0, 1, 1. 6 (LSB 0) + bit 1 → 7. 0 ve 1 atlanır. 7 (LSB 1) + bit 0 → 6. 4 + bit 1 → 5. 2 + bit 1 → 3. Stego: 7, 0, 1, 6, 5, 3. Alıcı 0 ve 1 olmayan katsayıların LSB'lerini okur: 1, 0, 1, 1. Değerler sadece (6,7), (4,5), (2,3) çiftleri içinde yer değiştirir; χ^2 testinin yakaladığı yapı budur [4].

Sunumda söyle. "Alıcı neyin atlandığını bildiği için aynı sırayla okuyabiliyor. Ama çift yapısı histogramda iz bırakıyor." (Sayılar örnek; kural [2].)

Hoca sorabilir.

- *Atlanan katsayıyı alıcı nasıl bilir?* Stego'da da 0 ve 1 olarak kalırlar, ve diğer değerler hiçbir zaman 0/1'e dönüşmez; alıcı 0/1'leri atlar.
- *Neden çiftler önemli?* LSB değiştirmek değeri çiftin diğer üyesine taşır; bu, POV problemidir [4].

Slayt 12: Outguess: scatter, then repair

Önce sen anla. v0.1: PRNG ile gömme konumlarını tüm DCT alanına saçar; χ^2 testi başta işe yaramadı ama istatistiksel iz bulundu [4]. v0.2: gömme sonrası istatistiği düzeltir (statistical restoration): cover'ı ikiye böl, birinde göm, diğerinde histogramı düzelt [2], [4]. Sorun: 8×8 blok sınırlarındaki süreksizlikle gömülen veri uzunluğu tahmin edildi [4]; sadece histogramı onarmak yetmez çünkü blok içi ve bloklar arası bağımlılıklar var, hatta tespit edilebilirliği artırabilir [2].

Sunumda söyle. "Outguess iki fikir getirdi: rastgele dağıtmak ve istatistiği onarmak. Onarım yalnızca birinci derece histogramı kapsıyor, blok yapısı bozulmaya devam ediyor."

Hoca sorabilir.

- *Restoration neden yetmedi?* DCT katsayılarının karmaşık bağımlılıkları var; ek bozulma bunları daha da bozar [2].

Slayt 13: F3: decrease, don't flip

Önce sen anla. F3 LSB'yi çevirmez, mutlak değeri 1 azaltır. Mesaj biti katsayıyla uyuşuyorsa değişiklik yok, değilse |katsayı| − 1 [4]. Bu JSteg'in POV sorununu çözer. Sorun: çift katsayılar fazla olur, histogram doğal görünmez [4]. Örnekler: 3 + bit 1 → 3; −2 + bit 1 → −1; 5 + bit 0 → 4; 1 + bit 0 → 0 (kaybolur, shrinkage'ın habercisi).

Sunumda söyle. "Amaç LSB çevirmenin izinden kurtulmak. Ama azaltma değerleri sıfıra doğru kaydırıyor ve çift değerler fazlalaşıyor."

Hoca sorabilir.

- *Azaltma neden çevirmeden iyi?* [2] yüksek tespit edilebilirliğin sebebini LSB çevirmenin kendisine bağlıyor.
- *Katsayı 0 olursa?* Alıcı sıfırları okumaz: shrinkage (slayt 15–16) [2].

Slayt 14: F4: the sign decides the bit

Önce sen anla. Bit değeri katsayının işaretine bağlanır: çift-negatif ve tek-pozitif = 1; tek-negatif ve çift-pozitif = 0. Bit sembole eşitse değişiklik yok, değilse |katsayı| − 1 [4]. Sonuç: histogram temiz görüntüye benzer [4]. Örnekler: 4 (sembol 0) + bit 1 → 3 (sembol 1); −2 (sembol 1) + bit 0 → −1 (sembol 0); 3 (sembol 1) + bit 1 → aynı.

Sunumda söyle. "F4, F3'ün histogram sorununu işaret tabanlı eşlemeyle çözüyor."

Hoca sorabilir.

- *F4'ün zayıflığı?* Makalelerde F4 için açık bir zayıflık yazmıyor [4]; F5'in matrix embedding getirmesini verimlilik hedefinin devamı olarak yorumladım (bu yorum benim).
- *Negatifler nasıl işleniyor?* İşaret eşlemesiyle: tek-negatif 0, çift-negatif 1 [4].

Slayt 15: F5: three ingredients and shrinkage

Önce sen anla. Üç parça: (1) |katsayı| azaltma, sadece sıfır olmayan AC [2]; (2) matrix embedding (Hamming) [2], [4]; (3) permutative straddling: mesajı tüm cover'a dağıtır [4]. Shrinkage: ±1 katsayı 0 olursa alıcı atlar, aynı bit sonraki katsayıya yeniden gömülür; verimlilik düşer, histogram etkilenir. F5 ±1'lerin yaklaşık %50'sini atlar [2]. −F5 beklenmedik şekilde F5'ten kötü; nsF5 (wet paper codes) yan bilgi kullanmayanlar arasında en iyi [2].

Sunumda söyle. "F5'in asıl yeniliği matrix embedding: aynı mesajı daha az değişiklikle gömmek. Shrinkage yan etki; nsF5 bunu wet paper codes ile çözüyor."

Hoca sorabilir.

- *−F5 neden kötü?* Makale bunu F5 tasarımcısının seçimini doğrulayan sürpriz sonuç olarak veriyor; sebep derinlemesine açıklanmıyor [2].
- *Permutative straddling nedir?* Mesajı tüm taşıyıcıya dağıtarak güvenliği artıran teknik [4].

Slayt 16: F5 shrinkage, step by step

Önce sen anla. Katsayılar 1, 3, 4; mesaj 0, 1 (matrix embedding olmadan gösteriliyor). Bit 0 → katsayı 1: LSB 1 ≠ 0, |1| − 1 = 0 (shrinkage). Alıcı sıfırı atlar, bit kaybolurdu; gönderici aynı biti sonraki katsayıya gömer: 3 → 2. Bit 1 → katsayı 4 → 3. Stego: 0, 2, 3. Alıcı 0'ı atlayıp 2 ve 3'ün LSB'lerini okur: 0, 1. Maliyet: 2 bit için 3 değişiklik.

Sunumda söyle. "Shrinkage'da bir bit için iki katsayıyı harcıyoruz, histogram da bozuluyor. Bu yüzden F5 ±1'lerin yarısını atlıyor, nsF5 ise wet paper codes kullanıyor." (Sayılar örnek; kural [2], [4].)

Hoca sorabilir.

- *Shrinkage neden sorun?* Alıcı sıfırı göremez, gönderici aynı biti tekrar gömmek zorunda; fazladan değişiklik ve bozulmuş histogram [2].
- *Gerçek F5'te matrix embedding burada nerede?* Gerçekte bitler gruplar halinde Hamming koduyla gömülür; burada shrinkage mekanizmasını sade göstermek için çıkardım.

Slayt 17: F5's key idea: matrix embedding

Önce sen anla. 7 katsayının LSB'sine 3 bit, en fazla 1 değişiklikle. H matrisinin sütunları 1'den 7'ye kadar sayıların ikili yazılışı [2]. Değişiklik yoğunluğu D = 1/(n+1), verimlilik E = (n+1)/n · k; (7,3) için 3,43 bit/değişiklik [4]. Bedeli: oran düşer (k/n azalır) [4]. Slayttaki sayılar: x = 1011001, m = 011, Hx = 100, Hx ⊕ m = 111 = 7. sütun, y = 1011000, Hy = 011.

Sunumda söyle. "Matrix embedding sendrom kodlamasıdır: mesaj, cover bitlerinin sendromu olur. Sendrom zaten mesaja eşitse hiç değişiklik gerekmez; değilse tek bit çevirmek yeter."

Hoca sorabilir.

- *Verimlilik neden artıyor?* Daha büyük k ile aynı tek değişiklik daha fazla bit taşır, ama hücre uzunluğu n = 2^k − 1 üstel büyür [4].
- *Neden güvenliği artırır?* Değişiklik sayısı azalır, yani değişiklik yoğunluğu düşer [2], [4].
- *Hiç değişiklik gerekmeme olasılığı?* 1/2^3 = 1/8 [2].

Slayt 18: Matrix embedding, step by step

Önce sen anla. Adımlar: (1) x = 1011001, m = 011, H 3×7. (2) Hx hesapla: satır 1 → 1, satır 2 → 0, satır 3 → 0, yani Hx = (1,0,0). (3) Hx ≠ m, değişiklik lazım. (4) Hx ⊕ m = (1,0,0) ⊕ (0,1,1) = (1,1,1) = H'nin 7. sütunu. (5) x'in 7. bitini çevir: y = 1011000. (6) Alıcı Hy = (0,1,1) = m. 3 bit, 1 değişiklik. Neden çalışır: 7. biti çevirmek Hx'i tam 7. sütun kadar değiştirir; fark da zaten o sütundu.

Sunumda söyle. Tabloyu satır satır oku; adım 4'te "fark H'nin bir sütunu olmak zorunda çünkü sütunlar tüm sıfırdan farklı 3 bitlik vektörler" de.

Hoca sorabilir.

- *Hx = m ise?* Hiçbir şey değişmez, y = x [2].
- *Neden her fark bir sütun?* H'nin sütunları 1..7'nin ikili yazılışı, yani tüm sıfırdan farklı vektörler [2].

Slayt 19: Hamming (7,4) and matrix embedding

Önce sen anla. Aynı H matrisi Hamming(7,4) kodunun parite-kontrol matrisidir (7 bit: 4 veri + 3 parite). Hata düzeltmede H·r sendromu hangi bitin yanlış olduğunu söyler; bit çevrilir ve hata düzelir. Steganografide ters yön: sendrom mesajdır; Hx ⊕ m bir sütunsa o bit bilerek çevrilir. Yani "hata düzeltme tablosu" "mesaj yazma tablosu" olur. Bu bölüm tezinle doğrudan ilgili.

Sunumda söyle. “[2] bunu binary Hamming codes olarak anıyor. (7,4) adı ve tek hata düzeltme özelliği makalelerde yok; genel kodlama teorisi, kaynak Hamming 1950.”

Hoca sorabilir.

- *Hamming(7,4) ile fark?* Hamming kodu hatayı düzeltir; burada hata yerine bilinçli tek bit değişikliği ile istenen sendromu üretiriz. (Makale dışı; Hamming 1950 ile destekle.)
- *Neden (7,3) diyorsun, (7,4) değil?* (7,3) steganografideki (n,k) gösterimi: n = 7 taşıyıcı bit, k = 3 mesaj biti [4]. 7 − 3 = 4 olduğu için Hamming(7,4) ile ilişkili.

Slayt 20: The same idea as a hash, cell (1,3,2)

Önce sen anla. Fan vd. aynı şeyi hash ile yazar: f(a) = 1 olan bitlerin indekslerinin XOR'u [4]. Örnek: a = 101, w = 11. f(a) = 1 ⊕ 3 = 01 ⊕ 11 = 10 (2). y = w ⊕ f(a) = 11 ⊕ 10 = 01 (1). 1. biti çevir: a′ = 001. Alıcı f(a′) = 3 = 11 = w. y = 0 ise hücreye dokunulmaz.

Sunumda söyle. “Matris çarpımı yerine tek bir hash: bitlerin indekslerini XOR'luyorum. Alıcı sadece hash'i hesaplıyor.” (Sayılar örnek; gösterim [4].)

Hoca sorabilir.

- *(1,3,2) ne demek?* d_max = 1 değişiklik, n = 3 bit, k = 2 bit mesaj [4].
- *Matris halindeki ile aynı mı?* Evet; hash, H'nin sütunlarının XOR'u olarak yorumlanır.

Slayt 21: Choosing p for a payload

Önce sen anla. p bit mesaj 2^p − 1 katsayıya gömülür; göreli yük α_p = p/(2^p − 1) [2]. p = 4, 5, 6, 7 için α = 0,267, 0,161, 0,095, 0,055. Yük (bpac) en fazla α_p olmalı. 0,1 bpac için p = 6 olmaz (0,095 < 0,1) ve p = 5 seçilir; 0,09 bpac eklenerek p = 6 etkisi görüldü [2]. Verimlilik değerleri (4,27 / 5,16 / 6,09 / 7,06) [4].

Sunumda söyle. “Yük düştükçe daha büyük p kullanabiliyorum ve değişiklik başına daha çok bit gömüyorum; bu yüzden küçük yüklerde matrix embedding daha çok işe yarıyor [2].”

Hoca sorabilir.

- *0,2 bpac neden p = 4?* α_4 = 0,267 ⩾ 0,2 ve α_5 = 0,161 < 0,2 [2].
- *Neden 0,09 eklenmiş?* p = 6 etkisini görmek için [2].

Slayt 22: Wet paper codes and STC

Önce sen anla. Wet paper codes: gönderici, alıcının bilmediği yan bilgiyi kullanabilir (hangi katsayılar değiştirilemez, "wet"). Dy = m, D paylaşılan rastgele ikili p×n matris; wet pozisyonlarda x_i = y_i. Hamming'in kesikli seçeneklerinden daha esnek, shrinkage'ı çözer (nsF5) [2]. STC: modern adaptif steganografinin standardı; "maliyet fonksiyonu + STC". msg = y·H^t, H köşegen boyunca küçük alt matrislerden (h × w) oluşur, Viterbi tarzı arama düşük maliyetli y'yi bulur; h genelde 8–12 [3].

Sunumda söyle. "Matrix embedding'in iki gelişmiş hali var: wet paper codes değiştirilemez konumları dışarıda bırakıyor, STC ise her konuma maliyet atayıp toplam maliyeti en aza indiriyor."

Hoca sorabilir.

- *Wet ve dry?* Wet: değiştirilmemesi gereken, dry: değiştirilebilir katsayı [2].
- *Maliyet fonksiyonu nedir?* Her katsayıyı değiştirmenin tespit edilebilirliğe etkisini ifade eden değer; örnek J-UNIWARD [3].

Slayt 23: STC and how errors spread

Önce sen anla. [3]'e göre stego bitindeki bir hata h mesaj bitini etkiler; bir mesaj biti h·w stego bitine bağlıdır. Her stego biti p_err olasılıkla yanlış, H elemanları 1/2 olasılıkla 1. İkili alanda XOR olduğu için mesaj biti sadece tek sayıda terim yanlışsa yanlış çıkar: p_mb = tek i için toplam C(hw, i)·(p_err/2)^i·(1 − p_err/2)^(hw−i). Sonuç: h ve w büyüdükçe hata olasılığı artar; w'yi küçültmek için gömme oranı düşürülür, h 8–12 seçilir.

Sunumda söyle. "Kayıplı kanalda tek bir stego bitindeki hata birçok mesaj bitine yayılıyor. Bu yüzden robust yöntemde hem oranı düşürüp hem hata düzeltme kodu ekliyoruz."

Hoca sorabilir.

- *Neden sadece tek sayıdaki hata?* İki hata XOR'da birbirini götürür (1 ⊕ 1 = 0) [3].
- *Neden w küçülünce iyi?* Bir mesaj bitine bağlı stego bit sayısı h·w azalır [3].

Slayt 24: How well do they hide?

Önce sen anla. P = ½(P_FA + P_MD): SVM tespit edicisinin asgari ortalama hata olasılığı. Yüksek = gizli, %50 = yazı-tura [2]. Gri seviye JPEG, QF 70, 6006 görüntü. 0,05 bpac'de: Steghide %2,58, MBS %4,61, JPHS %4,01, F5 %12,94, nsF5 %28,90, PQt %40,06, MM3 %40,86. Güvenli kapasite yaklaşık 0,05 bpac; matrix embedding güvenliği artırır; istatistik koruyan Steghide ve MBS en kolay tespit edilir [2].

Sunumda söyle. “En iyi yöntemler matrix embedding kullananlar. Ama bunlar 2007 sonuçları; daha yeni adaptif yöntemler (J-UNIWARD gibi) var [3].”

Hoca sorabilir.

- *P = %50 ne demek?* Dedektör rastgele tahmin yapıyor, yani tam gizlilik [2].
- *Neden Steghide tespit ediliyor?* [2]: istatistik koruma sadece birinci derece histogramı kapsıyor, diğer bağımlılıklar bozuluyor.
- *PQt ve MM3 neden iyi?* Gönderici sıkıştırılmamış cover'ı (yan bilgi) kullanır; PQ doku bilgisini de hesaba katar [2].

Slayt 25: Extension layers: which messages fit

Önce sen anla. 3 bitlik hücre sadece 4 durumu ifade edebilir (1 bit değişiklik veya değişiklik yok), bu yüzden 3 bitlik her mesaj gömülemez [4]. 1-layer: 00,01,10,11 kodları 000,010,100,110 olur (2 ile çarpım); 2-layer: 0000,0100,1000,1100 (4 ile çarpım) [4]. Kural: mod(w, 2^l) = 0, yani w sonunda l tane sıfır olur. Uyan yoksa standart matrix encoding'e dönülür [4].

Sunumda söyle. “Tablo, hangi uzun mesajların gömülebildiğini gösteriyor: sonu sıfırla biten mesajlar.”

Hoca sorabilir.

- *Neden sadece sıfırla biten?* Hash çıktısı 2^l ile çarpılan indeksler üzerinde kurulu; gömülebilen kodlar bu çarpım kümesi [4].
- *Rastgele mesajda?* Genişletme yalnızca yaklaşık 0,5 olasılıkla mümkün [4].

Slayt 26: Extended matrix encoding

Önce sen anla. Sorun: standart yöntemde n = 2^k − 1 zorunlu, k büyüdükçe oran çöker [4]. Çözüm: (d_max, n, k) üçlüsü (d_max, n, k, L) dörtlüsü olur. Alıcının katmanı bilmesi için hücreye m = ⌈log2(L+1)⌉ bitlik sembol eklenir. Sonuç: oran teorik olarak %100'ü aşabilir; ama k/n küçükken sembol bitleri yüzünden verimlilik F5'ten düşer; sembol için ortalama m/2 anlamsız değişiklik gerekir [4].

Sunumda söyle. “Kazanç mesajın yapısına bağlı; yazarlar siyah piksel dolu ikili logo kullanmış. Bu yöntemin dürüst sınırı.”

Hoca sorabilir.

- *Neden sembol gerekli?* Alıcı mesaja bakarak katmanı tahmin edemez [4].
- *Her durumda F5'ten iyi mi?* Hayır; k/n büyükken iyi, küçükken kötü [4].
- *Deney?* Lena 256×256, 64×64 logo, PSNR 70,33 [4].

Slayt 27: Extended encoding (1,7,3,2), step by step

Önce sen anla. Bu örnek [4]'te birebir var. a = 1101010, w = 11001. (1) mod(11001, 2^2) = 1 ≠ 0, sığmaz. (2) w = 1100: mod(1100, 2^1) = 0, katman l = 1. (3) m = ⌈log2 3⌉ = 2, s = 01. (4) 1 olan bitler 1, 2, 4, 6; indeksler ×2 = 0010, 0100, 1000, 1100; XOR = 0010 = f(a). (5) y = (1100 ⊕ 0010)/2 = 7; 7. biti çevir, a′ = 1101011; stego hücre c = 110101101. (6) Alıcı s = 01 → l = 1; a′daki 1'ler 1, 2, 4, 6, 7: 0010 ⊕ 0100 ⊕ 1000 ⊕ 1100 ⊕ 1110 = 1100 = w.

Sunumda söyle. Önce neden 11001 yerine 1100 denendiğini (sonunda sıfır yok), sonra sembolün neden eklendiğini (alıcı katmanı bilsin) anlat.

Hoca sorabilir.

- *Neden y'yi 2'ye bölüyoruz?* Hash 2^l ile genişletildiği için indeksi n aralığına geri indirmek gerekir [4].
- *Bu örnekte kaç bit gömüldü?* 4 bit (1100) 7 bitlik hücreye, 1 değişiklikle (+ sembol).

Slayt 28: Why the receiver always gets w

Önce sen anla. [4] Ek A: y. biti çevirmek hash'i tam o bitin değeri kadar XOR'lar. Gönderici y = w ⊕ f(a) seçtiği için f(a′) = f(a) ⊕ (w ⊕ f(a)) = w. y = 0 ise hiçbir şey çevrilmez ve f(a) zaten w'dir.

Sunumda söyle. "İspat tek satır: göndericinin seçtiği fark, hash'i tam mesaja taşıyor." Genişletilmiş halde indeks 2^l ile ölçeklenir, mantık aynı.

Hoca sorabilir.

- *y = 0 ise?* Hücreye dokunulmaz [4].
- *Neden XOR?* XOR kendi tersidir (a ⊕ a = 0), bu yüzden ek farkı eklemek hash'i tam hedefe çeker.

Slayt 29: Surviving JPEG recompression

Önce sen anla. Sosyal medya (Facebook, WeChat, Instagram) bant genişliği için JPEG'i yeniden sıkıştırır; bu geleneksel gömmeyi bozar [3]. DCRAS ve FRAS dayanıklı ama saldırı sıkıştırmasının QF'sini bilmeyi gerektirir (yan bilgi). Bao vd.: sanal cover bit = katsayı, komşu üç bloğun aynı konumdaki katsayılarının ortalamasından büyükse 1, değilse 0; STC ve J-UNIWARD tabanlı maliyetle gömülür; oran düşürülür, Reed–Solomon (6,15) eklenir. Sonuç: QF 65–85 arasında %0,3–3 bit hatası [3].

Sunumda söyle. "Mutlak değer yerine komşu katsayılara göre ilişki kullanıyorum; sıkıştırma değerleri değiştirse de ilişki daha kararlı. Bu, hocanın görüntü bozulması ile steganografi konusuyla doğrudan ilgili."

Hoca sorabilir.

- *Neden ilişkisel bit?* Sıkıştırma komşu katsayıları benzer şekilde etkiler, aralarındaki sıralama daha kararlıdır [3].
- *Bedeli?* Steganaliz karşısında DCRAS'tan en fazla %7 daha kötü; STC cover'ın sadece bir kısmında çalışıyor [3].

Slayt 30: Robust embedding, step by step

Önce sen anla. (1) JPEG'i çöz, 8×8 DCT blokları D_k(i). (2) M_ki = üç komşu bloğun aynı konumundaki katsayıların yuvarlanmış ortalaması. (3) Sanal cover biti: D ⩽ M ise 0, değilse 1. (4) Maliyet: J-UNIWARD maliyeti (değişiklik 1 veya 2), diğerleri çok büyük "wet" maliyet. (5) Düşük frekanslı konumlar, RS(6,15), STC ile gömme. (6) Geri yazma: bit 1 ise D = M + σ, bit 0 ise D = M − σ; σ = 1, 2 veya 3 (yerel farka göre; eşikler 15 ve 30). (7) Sıkıştırma sonrası: ortalamayı yeniden hesapla, D ⩽ M ise 0 oku, sonra STC ve RS çöz [3].

Sunumda söyle. Tabloyu gönderici (1–6) ve alıcı (7) olarak ikiye ayır. σ'nın amacı: değer, ortalamadan yeterince uzak kalsın ki sıkıştırma sonrası taraf değişmesin.

Hoca sorabilir.

- *σ neden var?* Sıkıştırma küçük kaymalar yaratır; M'den σ kadar uzak tutmak tarafın değişmesini azaltır (bu yorum benim, makale formülü [3]).
- *RS(6,15) ne?* Reed–Solomon hata düzeltme kodu; kalan bit hatalarını düzeltir [3].

Slayt 31: Beyond F5

Önce sen anla. Kapanış: üç yönün özeti. Genişletilmiş matrix encoding [4]: oran %100'ü aşabilir ama k/n küçükken verimlilik düşer. JPEG'e dayanıklılık [3]: sanal cover + STC + RS ile %0,3–3 hata. Derin öğrenme [1]: CNN ve GAN yöntemleri yüksek kapasite ve güvenlik sunar; ortak benchmark yok, GAN yakınsaması ve mode collapse açık problem.

Sunumda söyle. "LSB'den başlayıp tespit–düzeltme döngüsünü izledik: her yöntem bir öncekinin zayıflığını kapattı, kodlama teorisi (matrix embedding, STC) bu döngünün merkezinde." Sonra tezine bağla.

Hoca sorabilir.

- *Tezinde ne yapacaksın?* Cebirsel kodlama teorisi tabanlı gömme/kodlama yönünü derinleştireceğim (kendi planını buraya yaz).
- *En önemli ders?* Daha az değişiklik ve içeriğe duyarlı seçim güvenliği artırıyor [2].

Ek: Sunumda kullanabileceğin ekstra kaynak

- R. W. Hamming, “Error detecting and error correcting codes,” Bell System Technical Journal, 29(2), 147–160, 1950. (Hamming(7,4) adı ve hata düzeltme rolü için.)
- Makalelerin içindeki birincil kaynaklar (Westfeld F5, Crandall, Filler STC, Fridrich wet paper codes) için künyeyi ilgili makalenin kaynakça bölümünden al.